HW1 Problem Set

ECS 240

Due: Friday, October 23, 2026

This problem set contains 5 problems. Problems 1, 3, and 5 are coding problems, and problems 2 and 4 are pure theory problems. Problems 2, 3, and 5 build on Problem 1, while Problem 4 are standalone. Most problems have multiple parts. There is a template in `src/` which you can modify for the programming problems. Solutions to the theory problems should be written in LaTeX.

---

1. "Brainfuck" (we shall call it BF) is a toy programming language created by a Swiss computer programmer, Urban Müller, in 1993. It's known as one of the simplest "esoteric" programming languages, which means it is intended to be excessively simple and/or convoluted to write programs in, but it can be used to express arbitrary computation. It is also related to several other esoteric programming languages.

   There are eight commands in BF. The following table is reproduced from Wikipedia:[1]

Character	Semantics
`>`	Increment the data pointer by one (to point to the next cell to the right).
`<`	Decrement the data pointer by one (to point to the next cell to the left). Undefined if at 0.
`+`	Increment the byte at the data pointer by one modulo 256.
`-`	Decrement the byte at the data pointer by one modulo 256.
`.`	Output the byte at the data pointer.
`,`	Accept one byte of input, storing its value in the byte at the data pointer.
`[`	If the byte at the data pointer is zero, then instead of moving the instruction pointer forward to the next command, jump it *forward* to the command after the *matching* `]` command.
`]`	If the byte at the data pointer is nonzero, then instead of moving the instruction pointer forward to the next command, jump it *back* to the command after the *matching* `[` command.

   The syntax of BF can be any sequence of command characters, but with one important restriction: the left and right brackets (`[` and `]`) have to be well-matched. For example:

   ```
   [><+-[.]][+]
   ```

[1] https://en.wikipedia.org/wiki/Brainfuck

is a valid BF program, but

```
]><+[]
```

is not.

(a) Define the syntax for BF programs using a Rust enum, `enum BFProg`. You will want to have cases for the different syntax elements above, such as `IncData` for incrementing the data pointer, `DecData` for decrementing the data pointer, etc.

Hint/notes: There are multiple ways to define this. However, the important question is how to ensure the "left and right brackets are well-matched" requirement described above. In Rust - we can model this using one of the recursive cases (in our Rust enums). You should have only one recursive case of your Rust enum to correspond to the left/right bracket loops, such as a `Loop` case. You will also need a way to combine BF programs in sequence (like `Sequence` from class.)

(b) Give a way of printing out BF programs by implementing the `Display` trait for your `BFProg` data type. Display in Rust is a way to tell Rust how to print out a datatype as a string/text to the terminal.[2]

```
use std::fmt::Display;
impl Display for BFProg {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        // ...
    }
}
```

Your Display trait should print out the BF programs as a sequence of symbols, the way they are given above and in the Wikipedia examples. For example, they should print out like

```
[><+-[.]][+]
```

In your `main()` function, print out a couple of BF programs to see that they are displaying correctly.

(c) Write two unit tests for your syntax which attempt to construct a BF program for the two examples above:

```
[><+-[.]][+]
]><+[]
```

In your first unit test: assert that the first test program prints out correctly.

```
assert_eq!(prog.to_string(), "[><+-[.]][+]".to_string());
```

In your second unit test: you should be unable to construct that exact BF program; you may construct a related program and assert that it does *not* display as the string above.

```
assert!(prog.to_string() != "]><+[]".to_string());
```

(d) Define a reference interpreter for BF programs. This should be defined as a program `bf_interpreter()`. Use a similar code structure to our interpreter for the `RandProg` programs that we did in class.

[2] https://doc.rust-lang.org/std/fmt/trait.Display.html

```
fn bf_interpreter(prog: &BFProg) {
    // TODO
    unimplemented!()
}
```

(e) Write two unit tests which show that your reference interpreter is working correctly. The first should be an example BF program which adds two input numbers (like 5 + 6). The second one can be any example of your choosing.

```
#[test]
fn test_bf_interpreter_1() {
    unimplemented!()
}
#[test]
fn test_bf_interpreter_2() {
    unimplemented!()
}
```

2. This problem continues the previous problem.

(a) In LaTeX, give a formal grammar for the syntax of BF. You may want to review the definition of a context-free grammar and examples.[3] Your formal grammar should be based on, but not necessarily identical to, your Rust enum from problem 1.

(b) Prove that `]><+[]` is not a valid BF program given your formal grammar. (Note: this is importantly different from why this is not a valid BF program according to your Rust enum! In the Rust enum case, it should be impossible to construct this program; but in your syntax it should be because your grammar doesn't allow this string to be part of the grammar.

(c) Describe the semantics for BF programs by providing a *mathematical* definition of the set of triples `(INPUT, prog, OUTPUT)` that are valid executions of BF programs. You will need to start by defining INPUT and OUTPUT.[4] First: do this by assuming you have a deterministic reference interpreter, that is a program that can be used on any INPUT to run the BF program, and produces some output. You should use this to define a mathematical definition of the semantics.

Second part **(extra credit)**: Define this without assuming access to a reference interpreter of any kind, by directly defining the semantics. Here is a sketch: define a "configuration" of a BF program, define a starting configuration, and then define what it means for a configuration to step to another configuration. Then state when `(INPUT, prog, OUTPUT)` is valid by saying something like: there is a sequence of starting configurations $c_0, c_1, c_2, c_3, \ldots$, starting from INPUT on program `prog`, ... where you will need to relate each configuration with the previous one.

(d) Prove formally that the semantics of BF is deterministic. You should use your definition above and the definition of deterministic semantics given in class.

[3] The following may be helpful to see examples of syntax: https://cs.uwaterloo.ca/~s4bendav/files/CS360S22Lec08.pdf and https://en.wikipedia.org/wiki/Context-free_grammar

[4] **Hint:** sequence of bytes is a good choice.

3. This problem continues the previous two problems. Something we have not talked about yet in class is parsers. For this problem, you will write a naive parser for BF programs. Your parser should be implemented as a function `parse_bf`. It should take a Rust string reference (`&str`) that represents a BF program, and return either `Some(prog)`, if the program is a valid BF program, or `None`, if parsing failed.

   Here is a starting skeleton. Please use the signature as follows (you can modify the body).

```
pub fn parse_bf(bf_str: &str) -> Option<BFProg> {
    // define some additional state here ...
    // (what state would be useful to maintain during the parsing?)

    // Iterate through the string. We can use .chars():
    for char in bf_str.chars() {
        // TODO
        unimplemented!()
    }
}
```

   Write at least two unit tests that show your parser seems to be working.

   Answer the following in the LaTeX: is there any program for which displaying it, followed by parsing the resulting display, does NOT result in the same program that you started with? Why or why not?

   If yes, demonstrate an example of such a program in a unit test.

   Your unit tests for this part should be called `test_parser_1`, `test_parser_2`, `test_parser_3`, etc.

4. This problem will explore deterministic versus nondeterministic program semantics.

   Assume we have two programming languages, `Lang1` and `Lang2`. Assume that the `INPUT` and the `OUTPUT` in both languages is a sequence of integers. Let `Semantics1` and `Semantics2` be sets of `(INPUT, prog, OUTPUT)` pairs, so that

$$(\texttt{INPUT}, \texttt{prog}, \texttt{OUTPUT}) \in \texttt{Semantics1}$$

   if and only if the semantics oracle for `Lang1` accepts the program, and similarly for `Lang2`.

   We say that language 2 can *simulate* language 1 if there is a function f that takes any program p_1 in `Lang1` and produces a program $f(p_1)$ in `Lang2` such that, for all `INPUT` and `OUTPUT`:

$$(\texttt{INPUT}, p_1, \texttt{OUTPUT}) \in \texttt{Semantics1} \text{ iff } (\texttt{INPUT}, f(p_1), \texttt{OUTPUT}) \in \texttt{Semantics2}.$$

   (**Note:** iff means "if and only if".)

   (a) Prove that it is impossible for a deterministic programming language to simulate a nondeterministic one in this sense. That is, prove that if `Semantics2` is deterministic, and `Semantics1` is not, then `Semantics2` cannot simulate `Semantics1`.

(b) Prove, by giving a specific example, that the other way around is possible: that is, it is possible for a nondeterministic programming language to simulate a deterministic one.

(c) Prove that, despite part (a), it is possible for a deterministic programming language to simulate a nondeterministic one in the following different sense: if we are allowed to modify the input by adding additional integers to the `INPUT` sequence. The modified criteria for simulation is as follows: For all $\texttt{INPUT}_1$ and `OUTPUT`,

$$(\texttt{INPUT}_1, p_1, \texttt{OUTPUT}) \in \texttt{Semantics1}$$

iff there exists $\texttt{INPUT}_2$ such that:

$$(\texttt{INPUT}_2, f(p_1), \texttt{OUTPUT}) \in \texttt{Semantics2} \text{ and } \texttt{project}(\texttt{INPUT}_2) = \texttt{INPUT}_1$$

where above, `project` denotes an additional function defined by the simulation which takes $\texttt{INPUT}_2$ and gives back $\texttt{INPUT}_1$. The intuition should be as follows: $\texttt{INPUT}_2$ contains some additional input to the deterministic language, beyond what was present in the nondeterministic one.

5. This last problem will explore how deep mathematical statements can be embedded in program semantics.

   The *Goldbach Conjecture* is an open mathematics problem. It states that for any even integer greater than 2, that integer can be expressed as the sum of two prime numbers.

   Write a program in BF (or find one online) that terminates if and only if the Goldbach conjecture is false.

   Run your program in your BF interpreter (and check that it seems to not terminate). Ctrl-C your program after a while and make sure not to include the code which runs this program in your final submitted code.

   Then answer the following questions:

   (a) Do you think that this is anything special about the Goldbach conjecture? Or can all mathematical statements be expressed as whether a particular program terminates?

   (b) Give your best answer to the following: why is this not a problem for program semantics?

   That is, why is it not an issue that program semantics can express arbitrarily complex mathematical statements, that may be very difficult or even impossible to resolve? *(Note: we know there are some mathematical statements that cannot ever be proven one way or the other, although it is unknown if the Goldbach conjecture is one of such statements, it would seem to be plausible.)* Doesn't this make program semantics totally intractable as a subject? Yet, we can define program semantics for practical languages like C, Python, or Rust. Explain.

Submission instructions:

- Upload your solutions (as a PDF) and your code (as a `.zip` file, including the entire repository directory including config files like `Cargo.toml`, as with HW0) in Gradescope.
- Your PDF should be a single file `hw1.pdf`. If you use this LaTeX template to create your solutions, please remove the problem statements and include only your solutions.
- Your code (functions and types as described above) should be contained in `main.rs`. (You may also add utility modules, like `mod util;` then `use util::...`, if you feel that you need them; but you should put all the main functions with the names as above in `main.rs`.) Please double check on Piazza before importing any external crates (in your `Cargo.toml`).
- The `main.rs` should contain your solutions for parts 1, 3, and 5.
- Please include all of the function names and signatures as listed in the document above, and do not modify any function signatures. You are welcome to add additional functions and tests – but if you do so, please include all additional tests only at the bottom of the file.
- Make sure that `cargo build`, `cargo clippy`, `cargo test`, and `cargo fmt` (or `cargo fmt --check`) pass without any errors.